PROGRAMMING FOR PROBLEM SOLVING (BE01000121)

CHAPTER-10
FILE MANAGEMENT

ROADMAP

- File Operations: Opening, closing, reading, writing files.
- File Pointers: File pointers and basic file operations (for C).

INTRODUCTION

- **Why file handling in C…?**
- There are occasions when a program's output, after it has been compiled and run, does not meet our objectives. In these circumstances, the user would want to verify the program's output several times. It takes a lot of work for any programmer to compile and run the same program repeatedly.

FILE-HANDLING FUNCTIONS

- Basic file operation performed on a file are opening, reading, writing, and closing a file.

Syntax	Description
`fp=fopen(file_name, mode);`	This statement opens the file and assigns an identifier to the FILE type pointer `fp`. Example: `fp = fopen("printfile.c","r");`
`fclose(filepointer);`	Closes a file and release the pointer. Example: `fclose(fp);`
`fprintf(fp, "control string", list);`	Here `fp` is a file pointer associated with a file. The control string contains items to be printed. The list may includes variables, constants and strings. Example: `fprintf(fp, "%s %d %c", name, age, gender);`

FILE-HANDLING FUNCTIONS

Syntax	Description
`fscanf(fp, “control string”, list);`	Here `fp` is a file pointer associated with a file. The control string contains items to be printed. The list may includes variables, constants and strings. Example: `fscanf(fp, "%s %d", &item, &qty);`
`int getc( FILE *fp);`	`getc()` returns the next character from a file referred by `fp`; it require the `FILE` pointer to tell from which file. It returns `EOF` for end of file or error. Example: `c = getc(fp);`
`int putc(int c , FILE *fp);`	`putc()` writes or appends the character `c` to the FILE `fp`. If a `putc` function is successful, it returns the character written, `EOF` if an error occurs. Example: `putc(c, fp);`

FILE-HANDLING FUNCTIONS

Syntax	Description
`int getw( FILE *pvar);`	`getw()` reads an integer value from `FILE` pointer `fp` and returns an `integer`. Example: `i = getw(fp);`
`putw(int, FILE *fp);`	putw writes an integer value read from terminal and are written to the `FILE` using `fp`. Example: `putw(i, fp);`
`EOF`	`EOF` stands for “End of File”. `EOF` is an integer defined in `<stdio.h>` Example: `while(ch != EOF)`

FILE OPENING MODES

Mode	Description
r	Open the file for reading only. If it exists, then the file is opened with the current contents; otherwise an error occurs.
w	Open the file for writing only. A file with specified name is created if the file does not exists. The contents are deleted, if the file already exists.
a	Open the file for appending (or adding data at the end of file) data to it. The file is opened with the current contents safe. A file with the specified name is created if the file does not exists.
r+	The existing file is opened to the beginning for both reading and writing.
w+	Same as w except both for reading and writing.
a+	Same as a except both for reading and writing.

FILE OPENING MODES

Note: The main difference is w+ truncate the file to zero length if it exists or create a new file if it doesn't. While r+ neither deletes the content nor create a new file if it doesn't exist.

WRITE A C PROGRAM TO DISPLAY CONTENT OF A GIVEN FILE.

```c
#include <stdio.h>
void main()
{
    FILE *fp;
    char ch;
    fp = fopen("file1.c","r");
    do {
        ch = getc(fp);
        putchar(ch);
    }while(ch != EOF);
    fclose(fp);
}
```

WRITE A C PROGRAM TO COPY A GIVEN FILE.

```c
#include <stdio.h>
void main()
{
    FILE *fp1, *fp2;
    char ch;
    fp1 = fopen("file1.c","r");
    fp2 = fopen("file2.c","w");
    do {
        ch = getc(fp1);
        putc(ch, fp2);
    }while(ch != EOF);
    fclose(fp1);
    fclose(fp2);
    printf("File copied successfully...");
}
```

FILE POSITIONING FUNCTIONS

- `fseek`, `ftell`, and `rewind` functions will set the file pointer to new location.
- A subsequent read or write will access data from the new position.

Syntax	Description

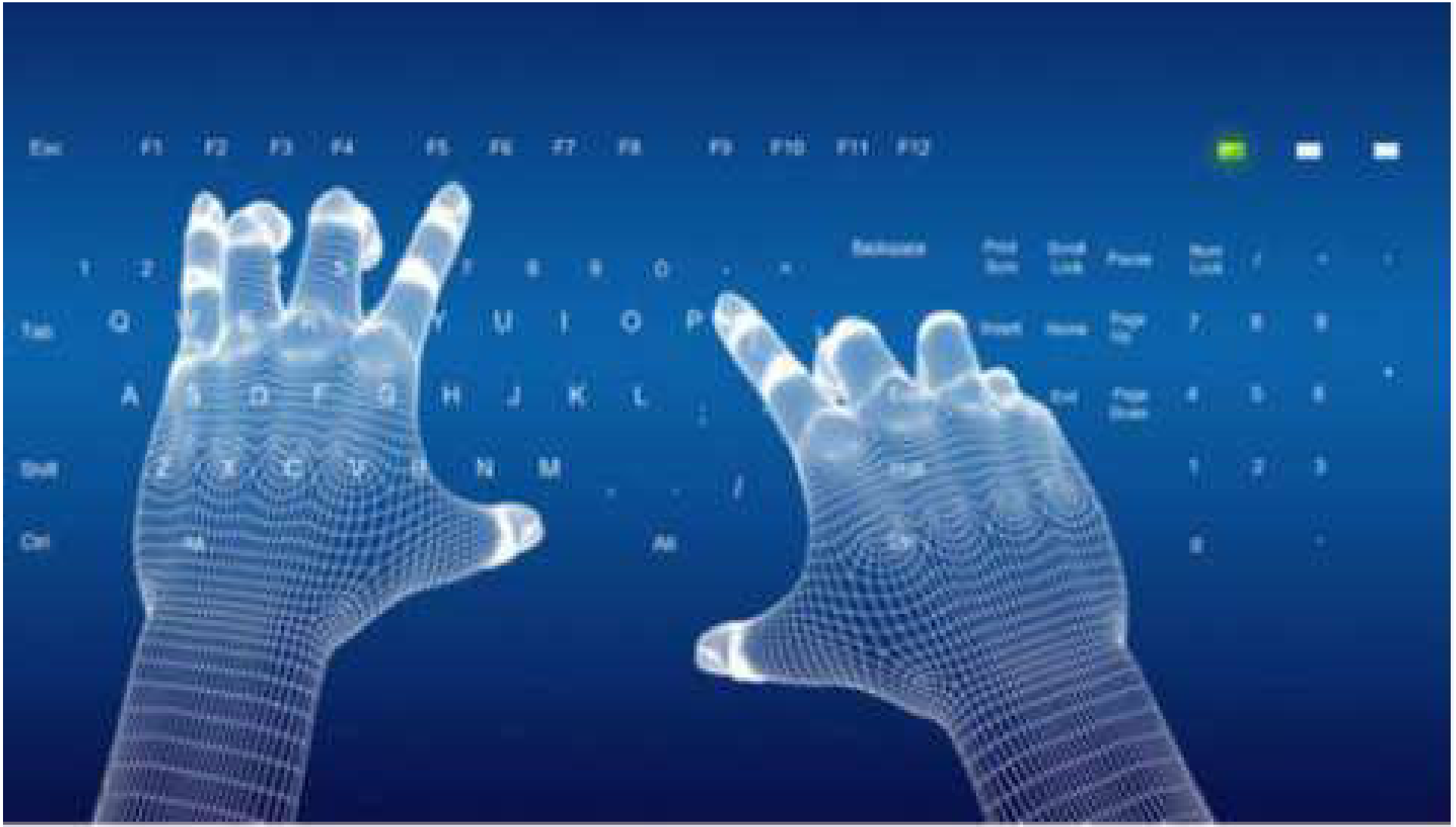
`fseek(FILE *fp,`
 	
`long offset,`
`int position);`
	`fseek()` function is used to move the file position to a desired location within the file. **`fp`** is a FILE pointer, **`offset`** is a value of datatype `long`, and **`position`** is an `integer` number.
	
	Example: /* Go to the end of the file, past the last character of the file */
	`fseek(fp, 0L, 2);`

FILE POSITIONING FUNCTIONS

Syntax	Description
`long ftell(FILE *fp);`	`ftell` takes a file pointer and returns a number of datatype `long`, that corresponds to the current position. This function is useful in saving the current position of a file. **Example**: /* n would give the relative offset of the current position. */ `n = ftell(fp);`
`rewind(fp);`	`rewind()` takes a file pointer and resets the position to the start of the file. **Example**: /* The statement would assign 0 to n because the file position has been set to the start of the file by rewind. */ `rewind(fp);`

WRITE A C PROGRAM TO COUNT LINES, WORDS, TABS, AND CHARACTERS

```
void main()
{
    FILE *p;
    char ch;
    int ln=0,t=0,w=0,c=0;
    p = fopen("text1.txt","r");
    ch = getc(p);
    while (ch != EOF) {
        if (ch == '\n')
            ln++;
        else if(ch == '\t')
            t++;
        else if(ch == ' ')
            w++;
        else
            c++;
        ch = getc(p);
    }
    fclose(p);
    printf("Lines = %d, tabs = %d, words = %d, characters = %d\n",ln, t, w, c);
}
```



Thank You !